

Nom:

Prénom:

BACCALAURÉAT GÉNÉRAL

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ

Session 2026

NUMERIQUE ET SCIENCES INFORMATIQUES

DURÉE DE L'ÉPREUVE : 2 heures

*Ce sujet comporte 6 pages numérotées de 1 à 6
La dernière page comporte une annexe que vous pouvez détacher.
Le candidat doit traiter les deux exercices proposés.*

L'utilisation d'une calculatrice n'est pas autorisée

Tournez la page S.V.P.

Exercice 1 ►

Cet exercice porte sur les graphes, les protocoles réseaux et la programmation orientée objet.

PARTIE A

Le graphe suivant modélise un ensemble de routeurs ; les sommets sont les routeurs, les arêtes les liaisons entre ceux-ci.

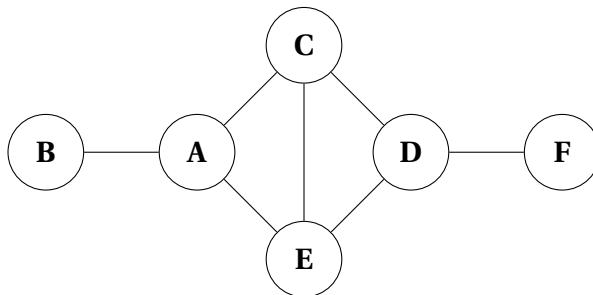


FIGURE 1 – Schéma des routeurs et des liaisons

On désire parcourir ce graphe en largeur depuis le sommet A. On considère que les sommets sont explorés dans l'ordre alphabétique.

1. Donner l'ordre de parcours des sommets du graphe.

Voici un résumé sommaire du fonctionnement du protocole RIP permettant à chaque routeur d'un réseau de taille modérée d'établir sa table de routage :

Règle a (règle d'initialisation). Chaque routeur initialise sa table en y ajoutant ses voisins directs. Ils sont accessibles en un saut, sans passer par aucun routeur intermédiaire.

Règle b (règle de transmission/réception). À intervalles de temps réguliers chaque routeur envoie sa table de routage à ses voisins.

Règle c (règle de mise à jour). Lorsqu'un routeur reçoit les informations d'un routeur voisin, trois cas peuvent survenir :

- une route vers un nouveau routeur lui est présentée : il l'ajoute à sa table de routage ;
- une route vers un routeur déjà connu lui est présentée, plus longue en nombre de sauts que celle inscrite dans sa table : elle est ignorée ;
- une route vers un routeur déjà connu lui est présentée, mais strictement plus courte en nombre de sauts que la précédente : l'ancienne est remplacée par celle-ci.

La réception par chaque routeur des tables de tous ses voisins et la mise à jour de sa table de routage en conséquence constitue une itération du protocole. Au bout d'un petit nombre de ces itérations, plus aucune table de routage ne varie, on dit que le processus est stabilisé.

Pour tout cet exercice, on n'envisagera pas les cas problématiques dans lesquels une liaison est coupée ou un routeur tombe en panne.

On considère des routeurs A, B, C, D, E et F connectés comme indiqué sur le graphe de la figure 1. Voici la table de routage de A à l'initialisation du protocole RIP :

Table de routage de A		
routeur	nombre de sauts	prochain routeur
B	1	–
C	1	–
E	1	–

2. Donner la table de routage de F à l'initialisation du protocole RIP.
3. Donner la table de routage de A après une première itération de RIP (deux réponses sont possibles).
4. Donner le numéro de l'itération de RIP à partir duquel les tables des routeurs du réseau ne varient plus.

On suppose dans la question suivante que les routeurs E et F sont reliés.

5. Donner la nouvelle table de routage de A après stabilisation de RIP (deux réponses sont possibles).

PARTIE B

Pour simuler la situation précédente et les tables de routage, on modélise le fonctionnement d'un routeur par la classe `Routeur`. Chaque instance `r` de la classe `Routeur` possède quatre attributs.

- `nom` : une chaîne de caractères qui identifie le routeur.
- `voisins` : une liste d'objets de type `Routeur`. Il s'agit de routeurs qui sont directement connectés au routeur `r`.
- `nb_sauts` : un dictionnaire qui associe à chaque routeur accessible depuis `r` le nombre de sauts nécessaires pour l'atteindre depuis `r`.
- `prochain` : un dictionnaire qui associe à chaque routeur `r_accessible`, accessible depuis `r`, le premier routeur sur un chemin qui mène à `r_accessible`, en `n` sauts, où `n` est la valeur associée à `r_accessible` dans `nb_sauts`. S'il y a un unique saut de `r` à `r_accessible`, alors la valeur associée au routeur `r_accessible` est `None`.

Initialement, tous les routeurs sont déconnectés : l'attribut `voisins` est initialisé avec la liste vide, et les attributs `nb_sauts` et `prochain` avec le dictionnaire vide.

</> Code Python

```
1 class Routeur:
2     def __init__(self, nom):
3         self.nom = nom
4         self.voisins = []
5         self.nb_sauts = {}
6         self.prochain = {}
```



Dans le programme principal, on crée les routeurs de la manière suivante :

</> Code Python

```
1 A = Routeur('A')
2 B = Routeur('B')
3 C = Routeur('C')
4 D = Routeur('D')
5 E = Routeur('E')
6 F = Routeur('F')
```



Ainsi que la liste des routeurs

</> Code Python

```
1 liste_routeurs = [A, B, C, D, E, F]
```

Afin de pouvoir relier les routeurs entre eux, on souhaite écrire une méthode `relie`, de la classe `Routeur`, dont on donne le code incomplet ci-dessous. Cette méthode prend en argument le routeur `self` ainsi qu'un routeur autre et met à jour si nécessaire les attributs `voisins`, `nb_sauts` et `prochain` des routeurs `self` et autre afin d'indiquer la présence d'une connexion entre ces deux routeurs. Dans le cas où les routeurs sont déjà connectés, cette méthode ne fait rien.

</> Code Python

```

1 def relie(self, autre):
2     if autre not in self.voisins:
3         self.voisins.append(...)
4         self.nb_sauts[autre] = ...
5         self.prochain[autre] = ...
6         if self not in autre.voisins:
7             autre.relie(...)

```

6. Compléter le code de la méthode `relie`.
7. Écrire la méthode `relie_liste` de la classe `Routeur` qui prend en paramètre une liste de routeurs `lst` et qui relie le routeur `self` à chacun des routeurs de la liste `lst`.

Par exemple, pour relier le routeur A aux routeurs B, C et E, on exécute l'instruction :

</> Code Python

```

1 A.relie_liste([B, C, E])
2 # On n'appelle pas B.relie(A) car la liaison est déjà faite

```

8. Écrire les instructions manquantes pour relier les routeurs de manière à obtenir le graphe de la figure 1.

D'après la règle c (règle de mise à jour) du protocole RIP, lorsqu'un routeur reçoit les informations d'un routeur voisin, il doit mettre à jour sa table de routage. On donne ci-dessous le code incomplet de la méthode `met_a_jour_table` qui implémente la règle c du protocole RIP.

</> Code Python

```

1 def met_a_jour_table(self, autre):
2     for r in autre.nb_sauts:
3         if r != self:
4             if (r not in self.nb_sauts or self.nb_sauts[r] > ...):
5                 self.nb_sauts[r] = ...
6                 self.prochain[r] = ...

```

9. Compléter le code de la méthode `met_a_jour_table` ci-dessus.
10. Écrire la méthode `itere_rip` qui prend en paramètre le routeur `self` et met à jour sa table de routage lorsqu'il reçoit la table de routage de chacun des routeurs présents dans la liste de ses voisins.
11. Écrire une fonction qui prend en paramètre une liste de routeurs `l_routeurs` et qui réalise une itération du protocole RIP pour tous les routeurs de `l_routeurs`.

Au bout de quelques itérations, le protocole RIP converge : plus aucune table de routage du réseau n'est modifiée. On aimerait pouvoir itérer le protocole dans le programme principal jusqu'à ce que ce soit le cas, à l'aide d'une boucle `while`.

On suppose que la méthode `met_a_jour_table` de la classe `Routeur` a été modifiée de telle sorte qu'elle renvoie `True` dans le cas où le routeur `self` a procédé à une mise à jour de sa table de routage, et `False` sinon.

12. Écrire une version modifiée du code de la méthode `itere_rip` de la classe `Routeur` de telle sorte que celle-ci renvoie `True` dans le cas où le routeur `self` a procédé à une modification de sa table de routage au cours de l'exécution de la méthode `itere_rip`, et `False` sinon.

On donne ci-dessous le code du programme principal. On suppose que les instructions permettant de relier les routeurs ont été écrites à la suite et que la situation est celle décrite dans le graphe de la figure 1.

</> Code Python

```

1 A = Routeur('A')
2 B = Routeur('B')
3 C = Routeur('C')
4 D = Routeur('D')
5 E = Routeur('E')
6 F = Routeur('F')
7 liste_routeurs = [A, B, C, D, E, F]
8 # instructions permettant de relier les routeurs

```

13. Compléter le code du programme principal afin que celui-ci mette à jour les tables de routage des routeurs présents dans la liste `liste_routeurs` jusqu'à ce qu'il ne soit plus nécessaire de faire des mises à jour des tables de routage.

On ne demande pas de réécrire les instructions permettant de connecter les routeurs entre eux.

Exercice 2 ►

Cet exercice porte sur les bases de données relationnelles et le langage SQL.

L'énoncé de cet exercice utilise les mots du langage SQL suivant :

DELETE, FROM, INSERT INTO, JOIN, SELECT, SET, UPDATE, VALUES, WHERE.

Les clés primaires seront soulignées et les clés étrangères seront précédées d'un #.

Le satellite GAIA a pour mission de cartographier un très grand nombre d'objets autour du Système Solaire. Régulièrement un catalogue est produit pour publier les données obtenues. Il est disponible sous différents formats dont par exemple sous forme de fichier csv. Num_Obj et identifie chaque objet cartographié de manière unique.

Num_Obj	Num_Systeme	Nom_Systeme	Type_Obj	Nom_Obj	Ascension_Droite	Declinaison	Parallaxe	Nom_SIMBAD
1	1	alf Cen	LM	Proxima Cen	217,392	-62,676	768,067	alf Cen C
2	1	alf Cen	Planet	Proxima Cen b	217,392	-62,676	768,067	
3	1	alf Cen	*	alf Cen A	219,902	-60,834	743,000	alf Cen A
4	1	alf Cen	*	alf Cen B	219,896	-60,838	743,000	alf Cen B
5	2	Barnard's Star	LM	Barnard's Star	269,449	4,739	546,976	Barnard's Star
6	3	Luhman 16	BD	Luhman 16 A	162,309	-53,318	501,557	Luhman 16 A
7	3	Luhman 16	BD	Luhman 16 B	162,308	-53,318	501,557	Luhman 16 B
9	5	Wolf 359	LM	Wolf 359	164,103	7,003	415,179	Wolf 359
10	6	HD 95735	LM	HD 95735	165,831	35,949	392,753	HD 95735
11	6	HD 95735	Planet	Lalande 21185 b	165,831	35,949	392,753	
12	7	alf CMa	*	alf CMa A	101,287	-16,716	379,210	alf CMa A
13	7	alf CMa	WD	alf CMa B	101,287	-16,721	374,490	alf CMa B
14	8	G 272-61	LM	G 272-61 A	24,772	-17,948	373,844	G 272-61 A
15	8	G 272-61	LM	G 272-61 B	24,772	-17,948	373,844	G 272-61 B
16	9	V1216 Sgr	LM	Ross 154	282,459	-23,837	336,027	Ross 154
17	10	HH And	LM	Ross 248	355,480	44,170	316,481	Ross 248

TABLE 1 – Données Astronomiques sur les Systèmes Stellaires Proches

Pour manipuler plus facilement les données, un chercheur utilise un système de base de données relationnelle, dans lequel il crée le schéma relationnel de la table Gaia :

Gaia (

```

Num_Obj : Int, Num_Systeme : Int, Nom_Systeme : String, Type_Obj : String,
Nom_Obj : String, Ascension_Droite : Real, Declinaison : Real, Parallaxe : Real,
Nom_SIMBAD : String)

```

PARTIE A

14. Justifier que l'attribut Num_Obj et peut être choisi comme clé primaire de la table Gaia.

Le type de l'objet (Type_Obj et) n'est pas une information directement compréhensible et le chercheur décide de créer une nouvelle table appelée Type. La clé primaire est Type_Obj et.

Soit la table Type suivante :

Type_Obj et	Libelle_Obj et
LM	Étoile de faible masse
Planet	Planète
*	Étoile
BD	Naine Brune
WD	Naine Blanche

TABLE 2 – Table Type

15. Proposer le schéma relationnel de la table Type en soulignant la clé primaire.

Dans ce qui suit, la table Gaia a été modifiée de manière à ce que le champ Type_Obj et soit une clé étrangère associée à la clé primaire de même nom de la table Type.

PARTIE B

16. Expliquer pourquoi le code SQL suivant ne fonctionne pas :

```
INSERT INTO Type VALUES ('BD', 'Trou Noir') ;
```

17. Indiquer le résultat de la requête suivante exécutée sur l'extrait présenté :

```
SELECT Nom_Obj et, Parallaxe FROM Gaia WHERE Type_Obj et = 'Planet';
```

18. Écrire la requête qui permet de récupérer le nom du système, le nom de l'objet et le libellé du type pour des objets ayant une parallaxe supérieure à 400 millisecondes d'arc et étant des 'Étoile'.
19. Le chercheur a besoin du nom, du libellé de leur type, de la déclinaison et de la parallaxe de tous les objets du système stellaire Alpha du Centaure ('alf Cen'). Écrire la requête qui permet d'extraire ces informations de la base de données.